

CARTEL: Consensus Adapting Real-Time and Efficient Logging

Seunghoon Lee, Wonyeong Lee, Seungyeon Cho, Seongtae Lee

Dept. of Computer Science & Engineering
Sungkyunkwan University
Republic of Korea
seunghoon.l@skku.edu

Jinkyu Lee*

Dept. of Computer Science & Engineering
Yonsei University
Republic of Korea
jinkyu.lee@yonsei.ac.kr

Abstract—Consensus algorithms are widely adopted in clustered systems to distribute data efficiently, with Raft being a prominent failover algorithm due to its effectiveness and fault tolerance. However, Raft and other consensus algorithms do not provide timing guarantees, limiting their application to time-critical systems such as industrial control systems, drone swarms, and autonomous vehicle networks, where it is critical to distribute state machines in time and prevent multiple timing misses of data.

In this paper, we propose **CARTEL** (Consensus Adapting Real-Time and Efficient Logging), a novel consensus algorithm that integrates time-predictability into the Raft framework. To achieve this aim, we examine Raft’s mechanisms and identify the characteristics that impact the timing of data propagation within a distributed system. Based on these insights, we design **CARTEL** by developing two mechanisms that address the primary limitations of Raft: (i) **CARTEL voting** to solve the uncertainty of leader election timing, and (ii) **CARTEL node buffer** to limit the number of indeterminately deferred data during leader failure. Moreover, we propose how to utilize the mechanisms of **CARTEL** to ensure time-predictability in leader elections and mitigate the indeterminate nature of data logging during leader transitions, without harming the integrity of Raft. We validate the effectiveness of **CARTEL** through real-world implementation. The experiments confirm that **CARTEL** not only reduces the uncertainty of system recovery inherent in the election process (by 65.7% compared to Raft) but also enhances the integrity of the distributed system by guaranteeing timely data logging.

I. INTRODUCTION

Modern real-time systems, including industrial automation, robotics, drone fleets, and autonomous vehicles, rely on distributed consensus algorithms to synchronize state across multiple nodes. These systems require strict timing guarantees to maintain safety, efficiency, and fault tolerance. A critical component of these systems is the consensus algorithm, which ensures that multiple nodes in a distributed environment agree on a common state despite failures and network partitions. For example, consider a robot cluster in a fulfillment center based on a consensus algorithm, as shown in Figure 1. Multiple robots form a distributed system, periodically sending data such as environmental sensor readings and task progress. A designated leader robot collects, logs, and propagates data to all system nodes. During this process, unpredictability and communication delays can lead to crashes or malfunctions

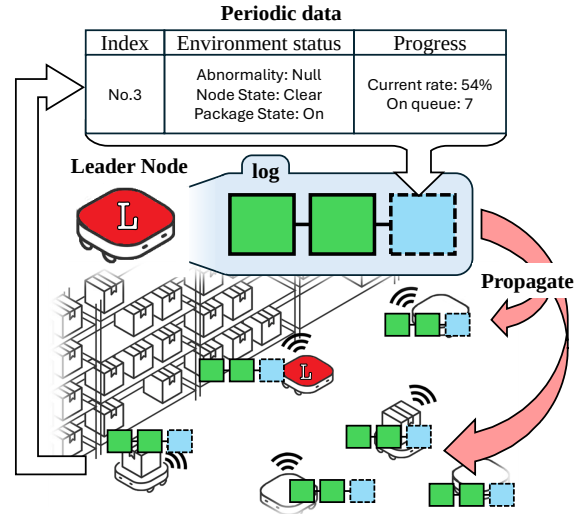


Fig. 1: Target system model: Fulfillment center

since the robots depend on timely data transmission. It is crucial to guarantee that the data sent from each node is logged and propagated within a specific time frame. Thus, the need for a consensus algorithm that values time constraints arises.

Among various consensus algorithms, Raft [1] has emerged as a widely adopted solution for partially synchronous fault-tolerant systems due to its simplicity and effectiveness in maintaining consistency and fault tolerance. However, when applied to time-sensitive distributed systems, such as industrial control systems in factories [2], drone swarm clusters [3], and MaaS (Mobility as a Service) networks comprising autonomous vehicles [4], Raft exhibits two major issues:

- F1. **Unpredictable Leader Election Time:** When a leader fails, Raft initiates an election process that can take an unbounded amount of time, especially during network delays or split votes. This lack of determinism is unacceptable in real-time applications that require bounded recovery times.
- F2. **Indeterminate Data Logging:** Raft relies on a single leader to handle all log updates. When the leader fails, in-progress data transmissions are lost or indefinitely delayed, leading to system instability and violating real-time constraints.

*Jinkyu Lee (corresponding author) initiated this work while affiliated with Sungkyunkwan University.

Recognizing these fundamental shortcomings, we introduce CARTEL (Consensus Adapting Real-Time and Efficient Logging), a consensus protocol that extends Raft with timing predictability at its core. Unlike prior optimizations that only tune existing mechanisms, CARTEL modifies the election and logging processes to eliminate uncertainty in failover timing and ensure uninterrupted data availability. CARTEL achieves this through two key mechanisms, CARTEL voting and CARTEL node buffer that integrate timing guarantees into the Raft framework, addressing F1 and F2, respectively, as follows.

To address the limitation of F1, we first analyze the response time of periodic data in fault-free conditions. We then examine the leader election process following a node failure and quantify the time required for a new leader to be elected. This analysis helps identify conditions under which data may be indeterminately delayed during re-election. As noted in F1, repeated split votes can theoretically cause unbounded election delays. We formalize the timing of such re-elections and derive the conditions that lead to failed elections.

Building on this insight, we propose a novel leader election algorithm, CARTEL voting, which ensures that the election process (and thus system recovery) is time-predictable and deterministic by requiring the majority of followers to send positive votes to a single candidate. Without compromising the integrity and robustness of the original Raft consensus algorithm, we derive a condition for nodes to select a candidate to which all followers can agree by utilizing the timing analysis from Raft. Since CARTEL voting succeeds in electing a leader in a single round, the timing guarantee of data becomes feasible.

Overcoming F2 involves examining the effects of leader failure on the system and analyzing the timing conditions that cause indefinite data delays. In time-critical systems, Raft’s reliance on a single leader compromises the ability to ensure timely data logging and propagation. To prevent the lack of predictability of data logging due to leader node failure, we introduce the CARTEL node buffer, which implements buffers on all nodes to store incoming data temporarily. When a follower detects the absence of a leader, it transitions to a candidate state and includes the buffered data in its vote request message. Follower nodes that vote for this candidate delete the data within their buffer and replace it with the data from the candidate. Once the candidate becomes the leader node and sends an `AppendEntries_RPC` (Remote Procedure Call), the followers transfer their buffer data to their logs, ensuring data preservation and system integrity.

To evaluate the performance, CARTEL was implemented in a real-use case environment using seven Jetson Orin Nano machines, and experiments were conducted. The machines are nodes that compose a cluster like Figure 1, where each node periodically sends data to the nominated leader node, and the data is propagated to every node within the cluster. The first experiment is aimed at observing the election of a new leader node after the failure of the former leader. In all cases, Raft completed the election within 8.80 seconds, while CARTEL took 3.40 seconds, significantly reducing the system recovery

time. We also investigated the occurrence and continuity of indeterminately deferred data across various periods in both consensus algorithms during a leader node failure. In Raft, there were up to 24 instances of continuous deferred data, posing critical risks to the system. In contrast, CARTEL did not experience deferred data; every periodic data point from seven machines was secured with predictable timing, demonstrating that CARTEL can satisfy timing guarantees for periodic data in distributed systems.

This paper makes the following contributions.

- We conduct the first systematic analysis of Raft’s timing unpredictability, revealing fundamental limitations in leader election and data logging for real-time systems (Section III);
- Through our analysis, we propose CARTEL, a novel fault-tolerant consensus mechanism based on Raft, which is the first timing guarantee of data commits (Section IV);
- We implement CARTEL voting to enable the failover process to be time-predictable (Section IV-A), and CARTEL node buffer to assure timely commit of data during the process of consensus (Section IV-B);
- We provide a formal timing analysis within CARTEL (Section IV-D) and discuss the integrity and limitations of CARTEL (Section IV-E); and
- We implement CARTEL on a cluster of edge devices, and evaluate the performance (in Section V).

II. FOUNDATION OF CARTEL: RAFT & SYSTEM MODEL

To develop a consensus algorithm that offers timing guarantees, it is essential to understand the characteristics and assumptions of Raft, the foundational protocol upon which CARTEL is built. Raft provides a leader-based consensus mechanism that ensures fault tolerance and consistency, making it a popular choice for distributed systems. However, its timing behavior—especially under failure scenarios—remains less explored. In this section, we present the operational principles of Raft and the system model we assume, laying the groundwork for the timing analysis and enhancements proposed in this paper.

A. Raft as the Underlying Consensus Framework

Raft is one of the most widely used consensus algorithms for practical deployments in partially synchronous systems closely resembling real-world environments. In such systems, there can be unpredictable communication delays and execution failures. However, after these transient phases, the system eventually stabilizes, allowing distributed algorithms to operate reliably and reach consensus. Raft is specifically designed for these conditions, using *timeouts* (e.g., *election timeout*, *heartbeat*) as timing assumptions to manage leader election, candidate selection, log replication, and fault tolerance.

Raft simplifies consensus through leader-based log replication. The system operates with three types of nodes:

- **Leader:** Handles all requests and replicates logs to followers. The leader node periodically sends *heartbeats*, which are empty `AppendEntries_RPCs` functioning as signals notifying nodes within the cluster of the leader’s

presence. The period of heartbeat is denoted as HB , which is randomly selected from $[HB^{\min}, HB^{\max})$.

- **Followers:** Passively accept log entries from the leader and acknowledge replication. Each follower maintains a timer called *election timeout* ET_X (where X is a node index), initially selected from $[ET^{\min}, ET^{\max})$ randomly and reset after receiving a valid RPC. Note that $ET^{\min}$ is always larger than $HB^{\max}$ and should also be configured to be at least a magnitude larger than the system's average broadcast time as specified in the Raft paper [1].
- **Candidates:** Arise from follower nodes when the leader node fails, initiating elections to select a new leader.

When a new item is submitted to the cluster, it is directed only to the leader, which appends the entry to its log and spreads it across the network by using `AppendEntries_RPC`. Followers acknowledge the `AppendEntries_RPC` from the leader, save the entry to their logs, and then send an acknowledgment of the entry back to the leader. When the leader receives acknowledgment from the majority of nodes within the system, the leader deems the entry committed. This process ensures that the committed entries saved in the state machine of every node are correct and safeguarded against loss, thus sustaining system integrity.

Another major function of Raft that ensures robustness is its reelection mechanism. When a leader potentially fails, the nodes within the cluster seamlessly initiate a new leader election, and a new leader emerges within a certain period. A follower node can recognize the leader node's failure when it does not receive an `AppendEntries_RPC` from the leader before its timer exceeds the election timeout and then transitions to the candidate state. The node then increases its term (a specific period during which a leader manages the cluster), assigns a new random election timeout, and initiates a new leader election. During this process, the candidate sends a packet called `VoteRequest_RPC` through all nodes to gather majority votes and becomes a leader when successful. `VoteRequest_RPC` holds the value of the current term, candidate ID, index of the last log, and the term of the last log. Followers will send a positive vote to a candidate and reset their election timer if it meets all of these conditions: i) followers have not already voted in the current term, ii) the term of the candidate is the same or higher than the followers', and iii) the candidate's log is at least as up-to-date as their own. Once elected, the leader assumes responsibility for orchestrating the entire system from log replication to maintaining consensus.

B. System Model and Timing Assumptions

Our system follows Raft's fault-tolerant consensus model, with robustness as a core principle. This feature holds even in the event of node failure or network partitioning. As long as the quorum, which is $\lfloor \frac{n}{2} \rfloor + 1$ nodes of the initial system (where n is the number of nodes within the system), remains operational, the system does not break and continues its operation; throughout the paper, we assume this necessary condition (i.e., maintaining an operational quorum), which is applied to

many consensus algorithms including Raft. Importantly, we do not account for Byzantine behavior—nodes are not assumed to act arbitrarily or maliciously.

We consider a distributed system with n interconnected nodes operating within a localized physical environment. The set of all nodes is denoted by $\mathbf{U}$, where each node is labeled as node_X , with X representing a unique identifier. The leader node is distinguished and denoted by node_Λ . Nodes within the cluster such as node_X periodically send data ϕ_X at intervals of T_{ϕ_X} . Let the y -th data ϕ_X be denoted as ϕ_X^y .

The underlying network is assumed to be predominantly stable, where communication among nodes is carried out over reliable point-to-point channels. Although transient communication instabilities may arise, the system is regarded as robust provided that the majority of communication links remain operational. We notate $\delta_{X,Y}^t$ as the network delay δ from node_X to node_Y that was sent from node_X at time t . $\mathbf{U}$ is used for the subscript of δ when transmitting/receiving to/from *every node* within the cluster, and Λ is used for that to/from *the leader*. For notational simplicity, we omit the subscript/superscript of δ whenever irrelevant.

A network delay threshold must be established for the system to operate with a timing guarantee. In a distributed system, the timing of successful data transmission is inherently uncertain due to various potential issues, such as node malfunctions or unstable networks. To ensure timing guarantees in such environments, the network delay threshold, which represents the maximum expected time for a message to be successfully transmitted, must be defined. Thus, we notate the network delay threshold as δ^* , and when data transmission exceeds this threshold due to any type of failures, it is assumed that the transmission is unsuccessful. Setting a network delay threshold aligns with previous studies that have provided upper bounds on network delays between nodes in different environments to guarantee timing [5]–[9].

Under these assumptions, we model the system as partially synchronous [10]. That is, the system may behave asynchronously for an unbounded yet finite duration, after which it eventually transitions to a synchronous regime.

III. TOWARDS TIME-PREDICTABLE RAFT: ANALYZING TIMING AND POSING CHALLENGES

In a system such as that illustrated in Figure 1, where periodic data generated by each robot is distributed across a cluster, both the integrity of the data and the timeliness of its arrival are critical to system's correctness and safety [11]–[13]. Although the Raft consensus algorithm has seen widespread adoption due to its simplicity and robustness, it remains insufficient in providing strict timing guarantees. This limitation arises primarily because a comprehensive analysis of Raft's consensus mechanism concerning time-predictability and data availability has yet to be conducted.

Accordingly, this section investigates Raft's internal mechanisms from the perspectives of timing behavior and data consistency. We begin by analyzing the data transmission process within the cluster under fault-free conditions. Subsequently,

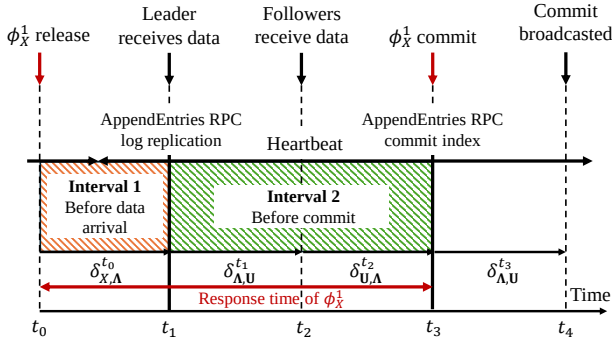


Fig. 2: Response time of data in Raft

we examine timing implications in the presence of failures that exceed the network delay threshold. Through this analysis, we identify key challenges that hinder Raft's ability to support time-critical systems.

A. Timing Analysis of Raft

To analyze Raft from a timing perspective, we focused on three functionalities: *log entry finalization*, *leader election*, and *split vote*. Recall Figure 1, within this system architecture, individual robots periodically transmit various data (environment status, progress state, etc.) to each other. This data serves as an essential input for each robot within the cluster, facilitating the operation and functionality of the entire system.

Log entry finalization. Consider the Raft process, where the data is stored in the cluster while the system is completely stable and no nodes fail, as shown in Figure 2. At t_0 , the first data ϕ_X^1 of node_X is released and sent to the system. After the network delay between node_X and the leader node of the distributed system $\delta_{X,\Lambda}^{t_0}$, the data is addressed to the leader at t_1 . When the leader receives a new log entry ϕ_X^1 , it sends `AppendEntries_RPC` to all nodes within the cluster $\mathcal{U}$ to notify ϕ_X^1 . All followers receive the entry by $\delta_{\Lambda,\mathcal{U}}^{t_1}$ later and replicate it in their logs. The followers then at t_2 respond with an acknowledgment containing the success or failure indicator flag to the leader. When the leader receives the responses from a majority of followers by at most $\delta_{\mathcal{U},\Lambda}^{t_2}$, it confirms ϕ_X^1 as committed and changes the state machine. From this specific time t_3 , since committed, ϕ_X^1 does not drop out of the consistent log of the system. We define the *response time* of ϕ_X^1 , denoted by $R_{\phi_X^1}$, as the period from its occurrence t_0 to its commitment t_3 . Shortly after the commit, the leader sends `AppendEntries_RPC` containing the information of the commit index to the network, where followers copy this information to their state machine.

Lemma 1: Suppose that data ϕ_X^1 of node_X is released at t_0 , and ϕ_X^1 will be sent from node_X to the leader node_Λ and then at t_1 be propagated to the majority of nodes by t_2 . Then, data ϕ_X^1 will be deemed to be committed at $t_0 + R_{\phi_X^1}$, where $R_{\phi_X^1}$ is as follows.

$$R_{\phi_X^1} = \delta_{X,\Lambda}^{t_0} + \delta_{\Lambda,\mathcal{U}}^{t_1} + \delta_{\mathcal{U},\Lambda}^{t_2} \quad (1)$$

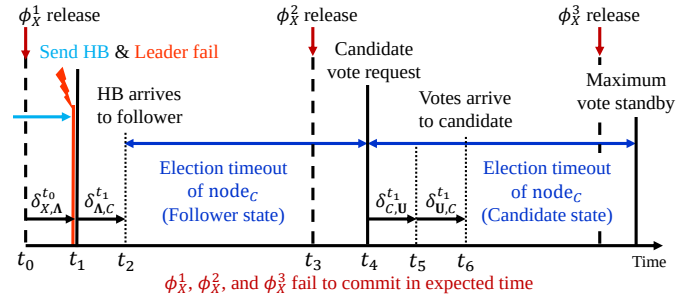


Fig. 3: Raft operation of leader election

Proof: For data ϕ_X^1 to be committed to the cluster, it undergoes three steps: 1) the data arrives to the leader, and the leader forwards the data to the followers; 2) the followers receive the data and respond to the leader; and 3) the leader commits the data once it receives a success response from the majority of the followers. Steps 1), 2) and 3) take the real network delay $\delta_{X,\Lambda}^{t_0}$, $\delta_{\Lambda,\mathcal{U}}^{t_1}$ and $\delta_{\mathcal{U},\Lambda}^{t_2}$, respectively, which proves the lemma. ■

Election. As mentioned in Section II, Raft tolerates node failures, including failure of the leader node. Here we analyze the election process that occurs when the leader fails from a timing perspective. Figure 3 illustrates a case where the leader fails shortly after sending its heartbeat to the nodes. The followers receive the last heartbeat of the leader after at most $\delta_{\Lambda,\mathcal{U}}$ and reset their election timer. One of the followers will recognize that the leader is missing since there is no additional `AppendEntries_RPC` from the leader during its election timeout. Thus, when its timer expires, the follower will change its state from a follower to a candidate, increase the term, and send `VoteRequest_RPCs`. The `VoteRequest_RPCs` goes to every node within the cluster; when a follower ensures that the candidate is valid, it comes back with a positive response message. The candidate waits for votes until the election timeout expires unless a `VoteRequest_RPC` with a higher term comes from another candidate within the cluster. The candidate that received votes from a majority of the nodes in the cluster for the current term becomes the new leader, and the system recovers from failure.

Lemma 2: Suppose the leader node_Λ fails at t_1 . Then, the election of a leader for the next term will conclude no later than $t_1 + R_E$, where R_E is as follows.

$$R_E = \delta_{\Lambda,\mathcal{U}}^{t_1} + 2 \cdot ET^{\max} \quad (2)$$

Proof: A follower becomes a candidate if the election timeout has passed since receiving the last `AppendEntries_RPC`. Considering the delay for this RPC to be transmitted from the leader to the follower, the maximum time for a follower to become a candidate after a leader's last `AppendEntries_RPC` at t_1 is $\delta_{\Lambda,\mathcal{U}}^{t_1} + ET^{\max}$ (by each election timeout upper-bounded by $ET^{\max}$). Furthermore, a follower that becomes a candidate sends `VoteRequest_RPCs` and waits for votes for up to $ET^{\max}$ within a single term. Therefore, Lemma 2 has been proved. ■

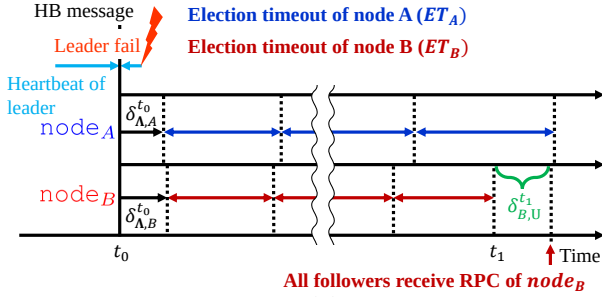


Fig. 4: Raft operation in a continuous split vote

In some cases, a candidate does not receive a majority of votes until their election timeout. This means that a split vote scenario occurred, where there were two or more candidates on that term, and votes from followers were divided evenly, thus no leader was elected. Note that ϕ_X^1 , ϕ_X^2 , and ϕ_X^3 of Figure 3 are lost or rejected for re-transmission during the election process. We will address this issue in Section III-B.

Split vote. The scenario of a split vote presents significant challenges in terms of timing analysis. The persistence of a split vote across multiple elections necessitates establishing data commit deadlines with a significant margin of safety to ensure reliability within a Raft-based system. From a temporal standpoint, the occurrence of a split vote significantly extends the system's recovery time and results in a continuous decline in periodic data throughput.

Example 1: Figure 4 exemplifies the scenario wherein a split vote perpetually recurs, necessitating repeated elections until a leader emerges. The randomly chosen values of election timeouts for each node make it difficult to predict when a leader will be successfully elected. In the worst case, the re-election process could continue indefinitely, potentially causing a system deadlock.

Upon these three Raft functionalities described above (i.e., log entry finalization, election, and split vote), we can derive a scenario where the leader election may continue indefinitely.

Lemma 3: Suppose the leader fails at t_0 , resulting in node_A and node_B becoming candidates for the N -th term, and node_A and node_B both receive the highest number of votes and neither obtains a majority. Then, the cluster may fail to elect a new leader only if each of the following cases satisfies its condition, as follows:

$$\begin{cases} \text{Case: } VR_A > VR_B, & \text{Condition of } 0 < \text{diff} < \delta_{B,U}, \\ \text{Case: } VR_A < VR_B, & \text{Condition of } 0 < \text{diff} < \delta_{A,U}, \\ \text{Case: } VR_A = VR_B, & \text{No condition.} \end{cases} \quad (3)$$

$$\text{where } \text{diff} = |VR_A - VR_B|, \quad (4)$$

$$VR_A = \delta_{A,A} + ET_A^F + \sum_{i=1}^N ET_{A,\pi}^C, \quad (5)$$

$$VR_B = \delta_{A,B} + ET_B^F + \sum_{i=1}^N ET_{B,\pi}^C. \quad (6)$$

Note that ET_X^F is the election timeout when node_X is a follower, and $ET_{X,\pi}^C$ is the election timeout of π -th term of node_X when it becomes a candidate.

Proof: While $VR_A > VR_B$, negating Lemma 3, suppose that $\text{diff} \geq \delta_{B,U}$ holds and VoteRequest_RPC of node_B is released no later than that of node_A by diff amount of time, and split vote occurs at N -th term. The term diff denotes the interval between the release of VoteRequest_RPC from node_A and node_B for the N -th term.

Without loss of generality, all the followers receive VoteRequest_RPC of candidate node_B no later than the VoteRequest_RPC of node_A . Due to the features of Raft, all nodes will vote for node_B and therefore do not result in a split vote, contradicting the supposition of the split vote occurring at N -th term. The same holds in the situation where $VR_A < VR_B$ with $\text{diff} \geq \delta_{A,U}$. ■

If Lemma 3 holds for every term, no leaders are elected, resulting in a system deadlock.

B. Timing Analysis with Network Timeout Failure

In distributed systems, data is often delayed or lost due to unpredictable network conditions. If a single data fails to reach its destination within a certain time threshold, the system may experience cascading failures, causing leader re-elections or loss of uncommitted data. In such cases, ensuring timely data commits to the system log becomes impossible, as delays may become indefinite due to specific node failures.

Definition 1: CARTEL defines situations where data and messages are not transmitted within the predetermined network delay threshold δ^* as *network timeout failure*.

Typically, when leaders or candidate nodes send messages to followers, network timeout failures do not have a crucial impact because the system can still function as long as the majority of nodes receive the message. However, as analyzed in Section III-A, when a leader fails and network timeout failures occur while transmitting messages or data to the leader node, the system itself can fail. In this subsection, we categorize and examine the causes of network timeout failures in two specific cases.

Case 1: Network timeout failure due to the malfunction of the leader node during the release and commit process,

Case 2: Network timeout failure during the leader election process.

Network timeout failure occurs when the leader fails during the response time for any data. However, we observe that the characteristics of network timeout failure of Case 1 differ when the leader fails during the following two specific intervals.

- *Interval 1:* the leader fails before the data arrives, and
- *Interval 2:* the leader receives the data but fails before committing it via log.

These intervals are represented in Figure 2 with orange and green hatches, respectively, and they exhibit the following properties.

Lemma 4: In Raft, if the leader fails during *Interval 1* of Figure 2, the timing of when ϕ_X^1 is committed to the system's log cannot be guaranteed.

Proof: When the leader fails during this period, since only the leader receives and stores data in its log for further propagation throughout the system, ϕ_X^1 does not reach the cluster, and node_X has to re-transmit the data after a certain period. The data is not saved in any of the nodes' logs, and even if the leader recovers from failure, it will not have the information of ϕ_X^1 in its entry. ■

Lemma 5: In Raft, if the leader fails during *Interval 2* (e.g., in Figure 2), there is no guarantee that ϕ_X^1 will not be overwritten.

Proof: During *Interval 2*, a leader-to-follower network timeout failure may occur due to network delays or leader failure before data transmission. In Raft, it is crucial to ensure that a majority of nodes in the cluster commit data. Without this majority acknowledgment, data remains vulnerable to being overwritten if it is not included in the newly elected leader's log. Once elected, the new leader replicates its log entries to followers and updates their logs, potentially replacing any conflicting entries. ■

Addressing Case 2, since data is transmitted regardless of the leader's presence, network timeout failure can also occur during the leader election process. For example in Figure 3, consider node_X sends ϕ_X^1 at t_0 . At $t_1 - \epsilon$ (where ϵ is a small time instant), the leader sends a heartbeat to the cluster and then fails, so the leader does not receive ϕ_X^1 . Following δ , followers receive the heartbeat message and reset their election timers. Once the node's election timeout (upper-bounded by $ET^{\max}$) elapses, it becomes a candidate. After an additional election timeout, the election for this term concludes. During the election, data sent to the cluster from node_X cannot be processed because there is no leader to send `AppendEntries_RPC`.

Lemma 6: Suppose the leader node node_Λ fails, and a split vote does not occur. Then, while periodic data ϕ_X is constantly transmitted within the network delay threshold δ^* , the number of data of ϕ_X that are not transferred to the leader node is at most L_{ϕ_X} , calculated as follows. Note that R_E^* corresponds R_E derived in Lemma 2, where the network delay threshold δ^* is used instead of the real network delay δ .

$$L_{\phi_X} = \left\lfloor \frac{\delta^* + R_E^*}{T_{\phi_X}} \right\rfloor, \text{ where } R_E^* = \delta^* + 2 \cdot ET^{\max}. \quad (7)$$

Proof: Suppose, for contradiction, that more than L_{ϕ_X} data items are not transferred to any leader. Consider the worst case where the leader fails immediately after sending its final `AppendEntries_RPC`, and a new data ϕ_X^α is transmitted just after. Since the network delay is bounded by δ^* , this data may not reach the leader before failure.

After the failure, the system enters a new election. From Lemma 2, the duration until a new leader is elected is upper bounded by

$$R_E \text{ in Eq. (2)} \leq R_E^* = \delta^* + 2 \cdot ET^{\max}.$$

Thus, any data transmitted during the interval of length $\delta^* + R_E^*$ —from just before the failure to just before the new leader starts processing—is not transferred.

Given the data period T_{ϕ_X} , the maximum number of such data items is L_{ϕ_X} in Eq. (7), which is applying the floor function for $\delta^* + R_E^*$ divided by T_{ϕ_X} . This contradicts the assumption that more than L_{ϕ_X} items were not transferred, which proves the lemma. ■

While data logging typically hinges on leader failure, a crucial distinction arises when considering data sent by the leader before failure via the `AppendEntries_RPC`. Even if such data is not yet committed, it persists in the followers' logs. Consequently, if a candidate, previously holding uncommitted data ϕ_X^1 , is elected as a leader and replicates its log to followers, ϕ_X^1 can be committed. However, in the context of timing guarantees, the probability of uncertainty of data commitment remains a concern, as any timing misses can be detrimental. Therefore, despite the potential for data logging under certain circumstances, the inherent risk of network timeout failure makes the timing guarantee of Raft challenging.

C. Timing Guarantees of Raft Consensus: Challenges

Despite widespread use in various fields requiring practical consensus algorithms, Raft, as analyzed in this section, presents several limitations when implemented in a distributed system in its naïve form. While node failures and system recovery are addressed in establishing a failover system with Raft, factors that influence system consistency, such as data availability and time criticality, are not guaranteed. Therefore, developing an efficient and safety-critical Raft consensus system poses the following challenges.

- C1. When a leader fails in Raft, the system initiates an election process to select a new leader for system management. Given Raft's application in distributed systems, network delays are common, often resulting in multiple candidates vying to be the elected leader. As detailed in Lemma 3, splitting of votes may occur, introducing uncertainty regarding the timing of the leader election. The absence of a leader inevitably halts system operation.
- C2. There is no certainty when and which data can be reliably committed to the system. While the nature of distributed systems inherently precludes perfect assurance of data dissemination to all nodes, the continuous miss of periodic data poses a significant challenge to many systems. As discussed in Section III-B, the failover mechanisms of Raft amplify this challenge by making it difficult to ensure timely data commits, to the point where such guarantees are no longer critical to system operations.

IV. CARTEL: TIME-PREDICTABLE RAFT CONSENSUS

We introduce CARTEL, a consensus algorithm that combines the timing guarantees with the practical benefits of Raft. This paper pioneers the integration of timing guarantees for data logging into the intricate failover consensus algorithms utilized within distributed systems. To address the challenges detailed in Section III, referred to as C1 and C2, we outline the goals for CARTEL as follows.

- G1. CARTEL prevents split votes that occur in Raft, making it possible to predict the time it takes to elect a new leader from the failure of the existing leader.
- G2. CARTEL secures the system from consecutive network timeout failures via a data-buffer mechanism and timing analysis results in Section III.

To address G1, we design CARTEL voting, which makes the election of a leader predictable in terms of timing, enabling timing guarantee for data log commits. For G2, we implement the CARTEL node buffer, temporarily allowing all system nodes to store incoming data during a leader node failure. However, relying solely on the CARTEL node buffer does not completely assure timely commit of data in complex failover scenarios. Accordingly, we derive a timing bound within CARTEL that ensures no consecutive network timeout failures occur for each periodic data interval.

A. CARTEL Voting

Lemmas 1–3 prove the uncertainty of when a leader will emerge. Moreover, Lemmas 4 and 5 prove that all data sent to the cluster during the election period cannot be guaranteed to be saved in logs within the system in time. With this insight, we design a method, CARTEL voting, to solve challenge C1.

In CARTEL voting, each candidate sends their election timeout value within `VoteRequest_RPC`. When receiving the first valid `VoteRequest_RPC` from a candidate, followers start their timer and wait for a period called *pending timeout*.

Definition 2: *Pending timeout* is a structured delay imposed on followers before voting in a leader election. This ensures they receive `VoteRequest_RPC` messages from all candidates, preventing premature votes and split elections.

When the timer for pending timeout expires, followers send a positive vote to the candidate with the shortest election timeout within the pool of `VoteRequest_RPC`s they received. The case of a split vote is minimized immensely since the majority of followers always vote for a single candidate. Thus, the followers act like a cartel, electing a candidate that suits their needs as the leader, leading to a fast recovery of the system and assuring the stability of the cluster.

Lemma 7: When multiple candidates participate in a leader election, the `VoteRequest_RPC` from the candidate with the shortest election timeout reaches all followers within $2 \cdot \delta^*$ of the first such message in the same term, assuming all network delays are bounded by δ^* .

Proof: Consider the worst case where the candidate with the shortest election timeout transmits its `VoteRequest_RPC` later than another candidate due to receiving the previous leader’s `AppendEntries_RPC` with maximum delay. The total latency in this case accumulates from the delay between the leader and the candidate, and then from the candidate to the follower.

Let t_0 be the time at which the previous leader sends its final `AppendEntries_RPC`, and $ET_{\min}^{\text{cand}}$ denote the shortest election timeout among the nodes that became candidates in the current term. In the worst case, this node receives the

leader’s message at time $t_0 + \delta^*$, waits for its timeout $ET_{\min}^{\text{cand}}$, and then sends a `VoteRequest_RPC` that takes up to δ^* to reach any follower. Therefore, the `VoteRequest_RPC` from this candidate arrives at most at:

$$t_0 + \delta^* + ET_{\min}^{\text{cand}} + \delta^*.$$

Meanwhile, the first `VoteRequest_RPC` to reach a follower could have come from a candidate node_X that received the leader’s message immediately at t_0 and delivered its `RPC` after a timeout of ET_X with negligible delay, arriving at time

$$t_0 + ET_X.$$

The arrival time difference is bounded by

$$(t_0 + \delta^* + ET_{\min}^{\text{cand}} + \delta^*) - (t_0 + ET_X) \leq 2 \cdot \delta^*.$$

Therefore, `RPC` from the candidate with the shortest timeout reaches all followers within $2 \cdot \delta^*$ of the first such message. ■

With the insight from Lemma 7, we demonstrate that split votes can be prevented by employing a pending timeout within CARTEL voting where nodes wait for `VoteRequest_RPC`s from every candidate node.

Theorem 1: Consider utilizing the pending timeout of CARTEL voting. If (i) and (ii) are satisfied, a leader is guaranteed to be elected in a single term without split votes:

- (i) The pending timeout of followers satisfies $2 \cdot \delta^* \leq \text{pending timeout} < ET_{\min} - 2 \cdot \delta^*$; and
- (ii) No two nodes have the same election timeout.

Proof: Assume, for contradiction, that a split vote occurs in a given election term despite both conditions (i) and (ii) being satisfied. That means two or more candidates each receive votes from a non-overlapping subset of followers, such that no candidate obtains a majority. For this to happen, at least one follower must have voted for a different candidate than the others. Since followers in CARTEL wait for a time window called *pending timeout* after receiving the first `VoteRequest_RPC`, they would have had time to receive all vote requests as long as *pending timeout* is no smaller than $2 \cdot \delta^*$, which holds by condition (i). Therefore, every follower sees all the candidates.

Given condition (ii), the election timeouts are all unique, so every follower will deterministically choose the same candidate—the one with the smallest timeout value. Thus, all followers vote identically, and a single candidate receives a majority. This contradicts the assumption that a split vote occurred. Therefore, no split vote can occur under the stated conditions, and a leader is elected in a single term. ■

Cost of determinism. Relative to Raft, which votes immediately upon receiving the first `VoteRequest_RPC`, CARTEL introduces a single explicit delay: the *pending timeout*. The window opens at the first `VoteRequest_RPC` arrival and closes before the follower casts its vote. This deterministic wait lets a follower consider all `VoteRequest_RPC`s that can arrive within the window, thereby preventing split votes

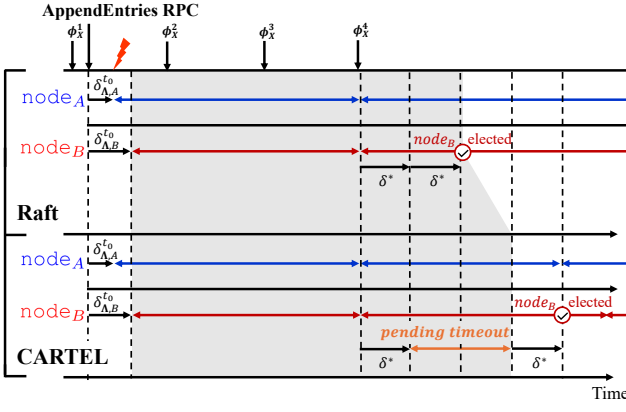


Fig. 5: CARTEL's cost of determinism

by construction. When Raft converges within a single election round under identical timers and link delays, it can elect a leader earlier than CARTEL by at most the *pending timeout*, as illustrated in Figure 5. This overhead is independent of the cluster size n .

B. CARTEL Node Buffer

Raft's uncertainty regarding the time at which data is committed to the log stems from its reliance on the leader for data transmission and storage. As explained in Section III-B, data from nodes are directed solely toward the leader. Consequently, network timeout failure occurs if the leader fails before data from the nodes are appended to the leader's log. To mitigate data loss during leader failover, CARTEL introduces a *node buffer*, denoted as buf_X , implemented in each node_X to uniformly receive and store incoming data from nodes within the cluster. This buffer temporarily stores data items released before they are appended to the follower's log via a valid `AppendEntries_RPC` from the current leader. By decoupling data reception from log-commitment, items generated after the last committed entry of a failed leader are not immediately discarded, increasing the likelihood that they will be preserved and committed once a new leader is elected.

The buffer operates according to the following rules:

- (i) Reception: Upon receiving a data item, the follower places it into its buffer unless it has already been committed to the log.
- (ii) Flush: Upon receiving a valid `AppendEntries_RPC` from the current leader, the follower atomically appends the corresponding entry to its log and clears its buffer.

These rules ensure that while the leader remains stable, all followers maintain consistent logs by periodically flushing their buffers after receiving the leader's heartbeats or RPCs.

To build intuition, Figure 6 illustrates how CARTEL node buffer operates.

Example 2: Suppose node_X periodically sends data ϕ_X ($\phi_X^1, \phi_X^2, \phi_X^3, \dots$) to the cluster. At t_0 , the leader issues its last `AppendEntries_RPC`, and node_C and node_F have stored ϕ_X^1 in their buffers. The RPC reaches node_C

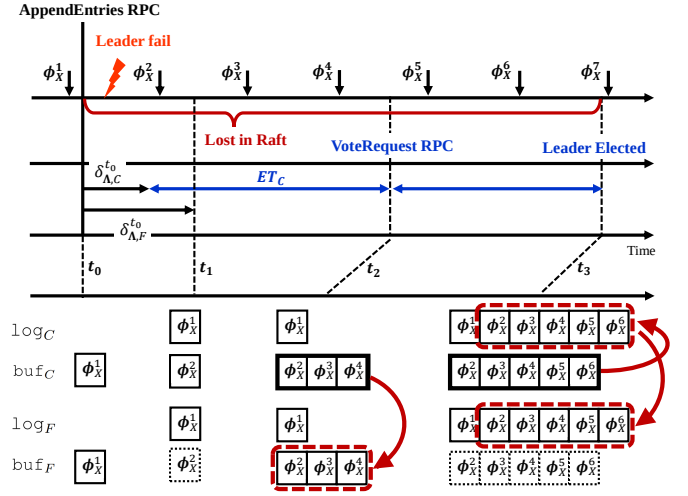


Fig. 6: Functionality of CARTEL node buffer

and node_F after delays (i.e., $\delta_{A,C}^{t_0}$ and $\delta_{A,F}^{t_0}$, respectively). Upon receiving the RPC, the nodes' buffers are flushed. In this scenario, when the `AppendEntries_RPC` containing ϕ_X^1 arrives, the nodes save ϕ_X^1 to their logs and flush their buffers, thus losing any buffered data that arrived before. At t_2 , node_C becomes a candidate and includes its buffered data in the `VoteRequest_RPC` sent to all nodes in the cluster. In response, node_F votes for node_C and copies the buffer data to its buffer. Subsequently, at t_3 , when node_C is elected as the leader, it saves all buffered data to its log and sends `AppendEntries_RPCs` to replicate the log across all nodes, maintaining the integrity of the system.

The node buffer does not compromise Raft's integrity: buffered data is only appended after a new leader is elected, thereby preserving the leader's append-only property. Furthermore, the flush rule ensures that the number of data items that can be lost due to a network-timeout failure in a single term is bounded, as formalized below.

Theorem 2: Consider that node_X periodically sends data ϕ_X to the cluster, where the period is noted as T_{ϕ_X} , and the cluster's network delay threshold is δ^* . Utilizing CARTEL node buffer within a single term can limit network timeout failures for data sent to the cluster following an `AppendEntries_RPC` to a maximum of N occurrences, even in the event of a leader's failure, where

$$N = \left\lceil \frac{\delta^*}{T_{\phi_X}} \right\rceil. \quad (8)$$

Proof: Let t_0 be the time the previous leader transmits its last valid `AppendEntries_RPC`. By the network-delay bound, each follower receives this RPC at some time in $[t_0, t_0 + \delta^*)$, denoted by the interval I . By the flush rule, each follower atomically appends the corresponding entry upon arrival and clears its buffer at its (node-specific) arrival time in I .

Any release of ϕ_X that can be purged by some follower's buffer flush must have been placed in that follower's buffer

before the flush occurs, i.e., before the arrival of the leader's last `AppendEntries_RPC` at that follower.

Since all arrival instants are located in I , the union of all such “purgeable” releases among followers is contained in the set of releases that occur within I . Therefore, the number of items that can be lost is upper-bounded by the number of release instants that fall in I .

Assume, for contradiction, that more than $N = \lceil \delta^*/T_{\phi_X} \rceil$ items are lost in this single term. Then there exist $K \geq N+1$ releases of ϕ_X with timestamps $t_0 \leq s_1 < s_2 < \dots < s_K < t_0 + \delta^*$. Because ϕ_X is strictly periodic with period T_{ϕ_X} , consecutive releases are separated by at least T_{ϕ_X} , hence

$$s_K - s_1 = (K-1)T_{\phi_X} \geq N \cdot T_{\phi_X} = \left\lceil \frac{\delta^*}{T_{\phi_X}} \right\rceil T_{\phi_X} \geq \delta^*.$$

However, $s_1, s_K \in [t_0, t_0 + \delta^*)$ implies $s_K - s_1 < \delta^*$, a contradiction.

Thus, at most N releases of ϕ_X can fall in I , and—by the observation above—at most N items can be purged by the post-`AppendEntries_RPC` flushes. Hence, the number of data items that can suffer network-timeout failure in this single term is bounded by N , completing the proof. ■

C. CARTEL in Action: Integrating Voting and Buffer for Predictable Consensus

Recall Figure 2, in Raft, data commit to nodes occurs only when the candidate is elected and sends `AppendEntries_RPC` as a leader. Implementing the CARTEL node buffer allows data to be temporarily retained. However, if a split vote delays leader election, this data remains only in followers' buffers and is not committed to their logs. The response time of data will be prolonged, exposing the system to various problems (e.g., network malfunction, node failure, etc.), and the timing guarantee of data that this paper pursues will not be met. Hence, although implementing a buffer prevents network timeout failure during election, it alone does not solve the challenge C2 in Section III-C.

CARTEL is completed when both CARTEL voting and CARTEL node buffer are met. Figure 7 compares the functionality of CARTEL to Raft in detail. Two nodes A and B acknowledge the absence of the leader, become candidates, and send `VoteRequest_RPCs` to nodes that arrive to all nodes by t_2 at the most. During $[t_2, t_3]$ of Figure 7(b), due to CARTEL voting, the followers that receive `VoteRequest_RPCs` wait for their pending timeout for other candidates' `VoteRequest_RPC` holding a smaller election timeout value. The followers do not store incoming data in their buffers during this process. Hence, ϕ_X^5 is only saved in the candidates' buffers.

The CARTEL mechanism's essence lies in utilizing the CARTEL node buffer during the CARTEL voting process to maximize the period during which the buffer can secure the data storage. If the `VoteRequest_RPC` of a candidate with a smaller election timeout comes, the follower vacates its existing buffer and replicates the buffer information contained within the `VoteRequest_RPC`. By t_3 , all followers have

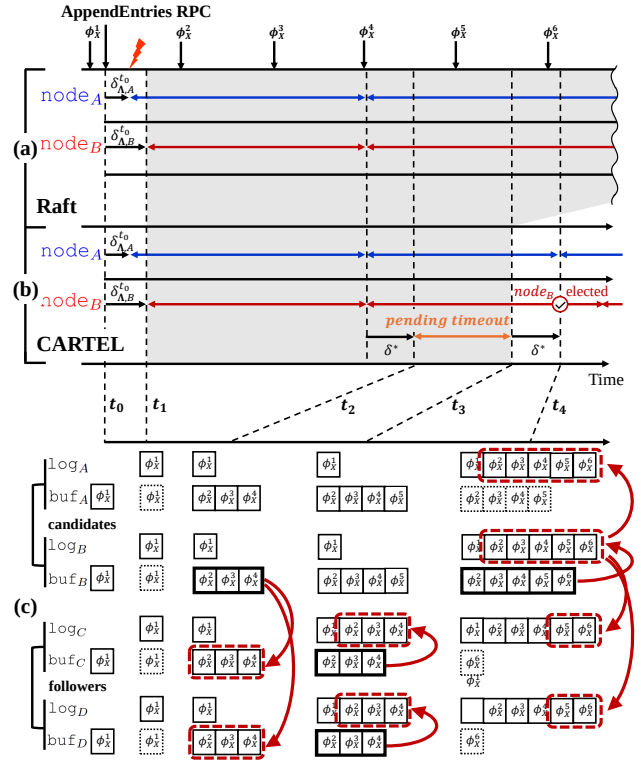


Fig. 7: Comparison of Raft and CARTEL: (a) Election period of Raft, (b) Election period of CARTEL, and (c) CARTEL mechanism

cast their votes for the candidate with the shortest election timeout. CARTEL voting ensures that all nodes vote for a single candidate, and the data within the RPC is stored in their buffers. Thus, in this case, ϕ_X^2 , ϕ_X^3 , and ϕ_X^4 are preserved by the majority. Even if such a candidate fails immediately after sending its `VoteRequest_RPC`, as data present in the majority of node buffers is regarded as committed (recall that in Raft, data stored in the majority's log is deemed committed), the subsequent failure of the candidate does not result in data loss. Although a re-election is triggered to select a new leader, the correctness and availability of the logged data remain unaffected, ensuring predictable commit behavior under such fault scenarios.

Therefore, it is plausible for the buffered data stored within the followers that sent votes to be moved toward their logs since the majority of nodes are holding the same data, ϕ_X^2 , ϕ_X^3 , and ϕ_X^4 in the case of Figure 7(c). With the CARTEL mechanism shown in Figure 7(b), data is successfully committed to the cluster within a fixed amount of time. This represents a significant improvement over Raft, illustrated in Figure 7(a), where both data commitment and leader election suffer from high uncertainty (highlighted gray in Figure 7).

To aid readability, we use Algorithm 1 to guide a step-by-step walkthrough of the failover process. Initially, it declares the inputs and per-node state and fixes L as the node with the smallest election timeout (Line 1). It then opens the follower-

Algorithm 1 CARTEL failover: election and post-election commit (single procedure; cf. Fig. 7)

Require: nodes; per-node election timeouts $ET[i]$; pending_timeout

- 1: **State** (for each node i): $\text{buf}[i]$, $\text{last}[i, \cdot]$, $L \leftarrow$ node with the smallest ET ; ties are broken by node ID
- 2: **procedure** CARTEL_FAILOVER
- 3: **for** each follower f **do**
- 4: $\text{first_request_time}[f] \leftarrow$ arrival time at f of the first valid `VoteRequest_RPC`
- 5: $\text{due}[f] \leftarrow \text{first_request_time}[f] + \text{pending_timeout}$
- 6: $\text{seen_cands}[f] \leftarrow \{c \mid \text{VoteRequest_RPC from } c \text{ arrived at } f \text{ before } \text{due}[f]\}$
- 7: $\text{chosen}[f] \leftarrow$ candidate in $\text{seen_cands}[f]$ with the smallest ET (tie $\rightarrow$ smallest ID)
- 8: schedule `Vote( $f \rightarrow \text{chosen}[f]$ )` at time $\text{due}[f]$
- 9: **end for**
- 10: *// Majority votes for a candidate //*
- 11: $\text{self_time} \leftarrow ET[L]$
- 12: $\text{times_at_L} \leftarrow [\text{self_time}]$
- 13: **for** each follower f with $\text{chosen}[f] = L$ **do**
- 14: append the reception time at L of `Vote( $f \rightarrow L$ )` to times_at_L
- 15: **end for**
- 16: sort times_at_L increasingly
- 17: $\text{majority_count} \leftarrow \lfloor \text{number_of_nodes}/2 \rfloor + 1$
- 18: $\text{election_time} \leftarrow \text{times_at_L}[\text{majority_count}]$
- 19: *// Leader announcement and commit on followers //*
- 20: leader L broadcasts `AppendEntries_RPC`
- 21: **for** each follower $i \neq L$ **do**
- 22: *// upon first valid AppendEntries_RPC: receive $\rightarrow$ append $\rightarrow$ flush*
- 23: **for** each sender $j \neq i$ **do**
- 24: $\text{new} \leftarrow \text{ARRIVALS_SINCE_LAST}(i, j, \text{now})$
- 25: $\text{buf}[i] \leftarrow \text{buf}[i] + \text{new}$
- 26: $\text{last}[i, j] \leftarrow \text{last}[i, j] + \text{new}$
- 27: **end for**
- 28: append $\text{buf}[i]$ to the log; $\text{buf}[i] \leftarrow \emptyset$
- 29: **end for**
- 30: **return** election_time
- 31: **end procedure**

side preselection loop (Line 3), where each follower f records the arrival time of its first valid `VoteRequest_RPC` as $\text{first_request_time}[f]$ (Line 4), sets the observation deadline $\text{due}[f] = \text{first_request_time}[f] + \text{pending_timeout}$ (Line 5), collects the candidates observed before that deadline in $\text{seen_cands}[f]$ (Line 6), deterministically chooses $\text{chosen}[f]$ as the smallest- ET candidate with ties broken by ID (Line 7), and schedules exactly one vote at $\text{due}[f]$ so that late or duplicate requests

are ignored by construction (Line 8). The procedure next transitions to majority formation at L (Line 11): it sets $\text{self_time} = ET[L]$ (Line 11), initializes times_at_L with this value (Line 12), and for each follower that selected L appends the actual reception time at L of `Vote( $f \rightarrow L$ )` to times_at_L (Lines 13–14); it then orders the collected arrival times (Line 16), computes the quorum size $\text{majority_count} = \lfloor \text{number_of_nodes}/2 \rfloor + 1$ (Line 17), and defines election_time as the majority-th arrival—i.e., the instant leadership takes effect (Line 18). Finally, it introduces the post-election stage (Line 19), broadcasts `AppendEntries_RPC` from L (Line 20), and, upon the first valid arrival at each follower, begins commit handling (Lines 21–22): the follower reconciles arrivals since $\text{last}[i, j]$ by staging them in $\text{buf}[i]$ and advancing the per-sender indices (Lines 23–27), and then applies the staged buffer to the log and clears it to realize commit-on-announce (Line 28). The procedure then returns election_time (Line 30).

Theorem 3: Let CARTEL be in effect, with both (i) CARTEL voting and (ii) CARTEL node buffers active. Suppose a follower node votes for a candidate node_C , and in doing so receives and stores buffered data ϕ from node_C . Then, if node_C is subsequently elected as leader, ϕ becomes committed and is logged in all followers that participated in the election.

Proof: Let us consider the following sequence of events in a CARTEL election cycle. Candidate node_C includes its local buffer content ϕ in the `VoteRequest_RPC` messages sent to all follower nodes. Each follower that votes for node_C replaces its buffer with ϕ from node_C . By CARTEL voting, the majority of nodes vote for the candidate with the shortest election timeout, and all votes converge to a single winner—candidate node_C . After receiving a majority of votes, node_C becomes the new leader. It then issues an `AppendEntries_RPC`, which confirms its leadership and triggers log replication. Upon receiving `AppendEntries_RPC`, each follower commits its buffer (which now contains ϕ), moves ϕ to its local log, and acknowledges the commit to the leader.

This process satisfies Raft’s commitment rule: A log entry is considered committed when it is present in the logs of a majority of the nodes. Since:

- The majority of nodes hold the same data ϕ in their buffer,
- And the elected leader replicates the buffer contents to all nodes via RPCs.

It follows that:

- The data ϕ is committed across the cluster,
- And logged in the majority of nodes.

Thus, any buffered data transferred via CARTEL voting is safely committed once the candidate becomes the leader. ■

D. Providing Timing Bound of Data in CARTEL

Although CARTEL demonstrates solid performance in guaranteeing the timing of data logging, there exists a scenario where the timing of data commitment remains uncertain,

meaning that the challenge C2 of Section III-C is not yet fully resolved. Specifically, if the candidate expected to be elected as leader fails during the interval $[t_3, t_4)$ of Figure 7, the timing of when ϕ_X^5 will be logged cannot be guaranteed. This is because the majority of followers have not accepted the new data into their buffers and have only saved the buffer data of candidates sent within `AppendEntries_RPCs`. ϕ_X^5 was only saved within the buffers of candidates, which is the minority within the cluster.

Recall C2 of CARTEL: since Raft is used in a partially distributed system, perfectly guaranteeing that all data will be saved in all logs within the system is nearly impossible. However, with the timing analysis of Section III, we can build a system that prevents consecutive network timeout failures by deriving the bound of the period as follows.

Theorem 4: When the period of ϕ_X satisfies $\delta^* < T_{\phi_X} < ET^{\min}$, then under CARTEL, no two consecutive data entries from `nodeX` experience network timeout failure—that is, no consecutive data losses occur during failover and leader election.

Proof: Suppose that two consecutive data entries ϕ_X^α and $\phi_X^{\alpha+1}$ are both lost due to network timeout failure (i.e., neither is committed to the log nor preserved in any follower’s buffer), although the period of ϕ_X satisfies $\delta^* < T_{\phi_X} < ET^{\min}$.

Let ϕ_X^α be transmitted to the cluster just before the last `AppendEntries_RPC` is sent by the leader prior to its failure. This timing represents the worst-case scenario for data preservation, as the data arrives too late to be committed yet just in time to be erased during the buffer flush triggered by the final RPC. Since CARTEL guarantees this RPC reaches all followers within the network delay threshold δ^* , each follower flushes its buffer upon receipt, removing any uncommitted data, including ϕ_X^α . Therefore, ϕ_X^α is lost from the system.

Now consider $\phi_X^{\alpha+1}$, which is transmitted after an interval $T_{\phi_X} > \delta^*$ (by the supposition) and thus arrives after the buffer flushes. It is therefore freshly stored in the buffers of followers. Since $T_{\phi_X} < ET^{\min}$ holds by the supposition, the data arrives before the leader election concludes. Under CARTEL’s election mechanism, at least one follower holding $\phi_X^{\alpha+1}$ in its buffer becomes the candidate and is elected as the new leader. As a result, $\phi_X^{\alpha+1}$ is preserved and committed through subsequent log replication. This contradicts the supposition. ■

With the mechanisms of CARTEL and period bounding, we build a system that commits data to the log in a reliable manner, effectively addressing challenge C2.

E. Safety and Trade-offs in CARTEL

The mechanisms of CARTEL voting and CARTEL node buffer make CARTEL work differently from Raft but keep the safety¹ as a leader-based consensus algorithm, as follows. While the candidate with the shortest election timeout is selected as the new leader, the regulations governing candidate elections (e.g., term-based integrity, up-to-date log entries) are

¹In Raft [1], safety ensures that once a value is committed to the log, it will remain intact and unaltered, preserving the system’s integrity.

inherited from Raft, which ensures integrity. The CARTEL buffer acts as an intermediary layer, ensuring that even if the leader fails before data is received, other nodes in the system can retrieve and preserve that data. Most importantly, it does not compromise the Raft’s leader append-only rule. Data from the buffer is only copied to the log after a new leader is officially elected. Furthermore, while the leader remains intact, it corrects any inconsistencies in followers’ logs—such as those that may have voted for the wrong candidate—by overwriting the incorrect entries and maintaining log consistency.

Meanwhile, CARTEL introduces several trade-offs and limitations. During the leader election process, candidates transmit buffered data to all nodes, increasing network traffic and potentially causing congestion in larger systems. Furthermore, node buffers require additional storage capacity, which may pose challenges in systems with stringent resource constraints.

The maximum size of a CARTEL buffer can be bounded by the amount of data generated during the period in which the buffer is not flushed. Let $T^{\min}$ denote the minimum data generation period among all data sources in the system, and $ET^{\max}$ the maximum election timeout. As shown in Fig. 7, the longest such buffering interval occurs between the last valid `AppendEntries_RPC` from the previous leader and the point at which a new leader is elected, during which buffered data remains uncommitted. Since the leader election duration is upper-bounded by $2 \cdot ET^{\max}$, the maximum number of buffered entries per node (denoted by $B^{\max}$) is bounded by

$$B^{\max} = \frac{2 \cdot ET^{\max}}{T^{\min}} \times n, \quad (9)$$

where n is the number of nodes.

While the total number of buffered entries may increase as the system scales (e.g., with more data sources or higher data rates), this growth remains predictable and thus can be provisioned for in advance. In the context of our experimental platform (NVIDIA Jetson Orin Nano), the resulting buffer sizes are well within practical memory limits. Although, as noted above, scalability concerns may arise in microcontroller-based systems with severe memory constraints, such concerns are largely mitigated by the fact that the buffer size can be bounded as described above.

V. EVALUATION AND EXPERIMENTS

In this section, we evaluate the performance of CARTEL by experimenting with it in real-world settings.

Implementation and experimental settings. The goal of CARTEL is the practical use of the fail-over consensus algorithm in a distributed system. We imitate one of our target scenarios, industrial control systems of factories, and build a distributed system of edge devices. We use seven NVIDIA Jetson Orin Nano 8GB Module devices to configure our experimental environment. CARTEL was programmed with Python 3.10 and was run on Ubuntu 22.04. During experimentation, leader failure is performed by shutting down the Jetson Orin machine serving as the leader node at random intervals.

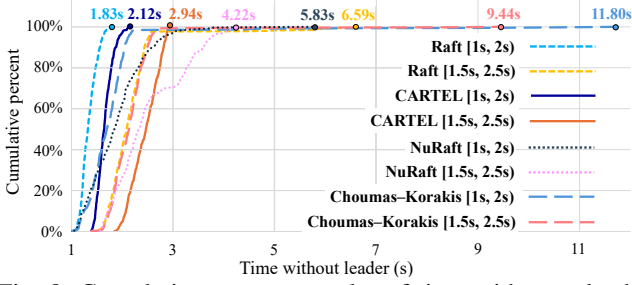


Fig. 8: Cumulative percentage plot of time without a leader

CARTEL uses a network delay threshold, denoted as δ^* , to ensure system integrity. We experimented to measure the actual broadcast time—the time taken for a signal to be sent and received—by sending signals between nodes 2 million times. The average broadcast time (the round-trip time) between nodes was 59.4 ms, with a maximum time of 110 ms. Based on these experimental settings, the network delay threshold² for a one-way trip, δ^* , was set to 100 ms ($\gg \frac{110}{2}$ ms). As the election timeout must be at least an order of magnitude larger than the broadcast time according to Raft [1], the election timeout in CARTEL is determined by the network delay threshold δ^* and is encouraged to be set an order of magnitude larger to maintain system stability. It is important to note that this network delay includes computational delays, as each device processes messages using Python.

Leader election. We evaluate leader election latency by measuring the time required for the system to elect a new leader and recover the system after the current leader fails. Within these cases, we compare the performance of Raft [1], NuRaft [14], approach of Choumas–Korakis [15], and CARTEL across scenarios where the election timeouts of nodes are randomly selected within specific ranges. In each case, we simulate a distributed system using seven Jetson Orins, introducing random failures to the leader node. Each experiment was repeated 500 times. Figure 8 shows the empirical CDF of the time without a leader for representative timeout intervals; the numeric labels at the tails denote the maximum observed values across the 500 trials.

We keep the timeout range fixed at 1s and compare the cases where election timeouts are [1s, 2s), [1.5s, 2.5s), and [2s, 3s). Figure 8 displays the [1s, 2s) and [1.5s, 2.5s) cases, while the [2s, 3s) results follow the same trend and are reported below. Generally, CARTEL completes elections faster than Raft in worst-case scenarios. However, in the [1s, 2s) timeout range shown in Figure 8, Raft appears to recover faster—its worst-case reaches 1.83s—but only under the assumption that no split votes occur. As discussed in Section IV-A, CARTEL pays an explicit cost for determinism via the *pending timeout*. This apparent advantage masks a critical vulnerability: Raft provides no mechanism to prevent vote splitting and is therefore susceptible to unbounded election delays. CARTEL, by comparison, exhibits a 2.12 s worst case; the pending win-

dow slightly extends election time under ideal conditions yet eliminates split votes altogether, yielding consistent, bounded-time failover—a guarantee Raft cannot offer. In real-time systems, where even rare outliers are unacceptable, such unpredictability makes vanilla Raft unsuitable without additional mechanisms.

To further validate the effectiveness of CARTEL in ensuring predictable leader election, we compare it with two recent enhancements to Raft’s election process: (i) NuRaft’s priority-based leader election [14], and (ii) the adaptive timeout approach of Choumas and Korakis [15]. NuRaft assigns a priority value to each node and adjusts the target priority dynamically so that higher-priority nodes are more likely to become the leader. While this heuristic reduces randomness and can improve average election time, it does not eliminate split votes and lacks a deterministic bound on leader election time—multiple rounds are still possible under adverse timing conditions. The Choumas–Korakis approach analytically models election timeout selection and adapts it to minimize the expected leader election latency given network conditions. However, it remains probabilistic by design, as it retains randomized timeouts and offers no guarantee against worst-case delays due to split votes.

As shown in Figure 8, both NuRaft and Choumas–Korakis improve the median failover time compared to Raft, but their distributions still include significant outliers. In the [1s, 2s) interval, NuRaft’s worst case is 5.83s, while Choumas–Korakis reaches as high as 11.90s. For [1.5s, 2.5s), the corresponding worst-case values are 4.22s and 9.44s, respectively. In contrast, CARTEL consistently achieves bounded recovery time of 2.12s and 2.94s for the two cases, with no occurrence of delays beyond its analytical bound. These results demonstrate that while existing optimizations improve the average case, only CARTEL ensures time-predictable worst-case behavior.

Data logging. In this subsection, we examine and compare timely data logging in Raft and CARTEL. The experiment is conducted under the same environment as the leader election experiment, using seven Jetson Orin Nano as nodes for the distributed system, where each node periodically sends data that has to be propagated to all and logged within the system. The election timeout interval is set to [1.5s, 2.5s). Each of the seven nodes sends data to the leader at intervals of 0.33s, 0.53s, 0.72s, 0.91s, 1.1s, 1.3s, and 1.49s, chosen with a difference of approximately 0.19–0.2s between each interval for evaluation purposes. We record whether data is logged from the time of the leader’s failure to the subsequent election.

The results are illustrated in Figure 9(a). We observe in Raft that data not logged within the expected time, as network timeout failure occurs, decreases as the period of data increases. The number of maximum network timeout failures in a single term was 24 when the period was 0.33s. This observation is demonstrated in Eq. (7) of Lemma 6, which shows that as the period T_{ϕ_x} decreases, the number of data not committed in time (upper-bounded by L_{ϕ_x}) increases significantly. The average number of data that did not transfer on time in Raft is

²Any messages or data arriving after this threshold are not guaranteed to be processed in a timely manner.

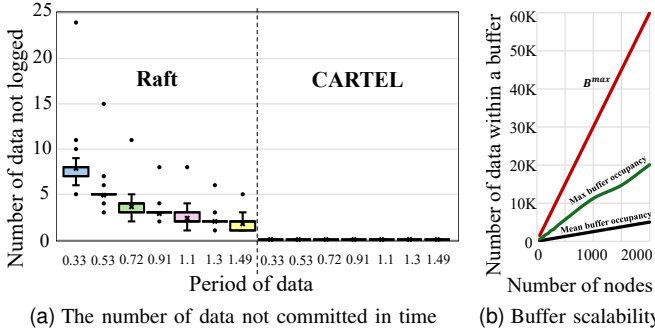


Fig. 9: Experiment and simulation results

7.85, 4.89, 3.60, 2.85, 2.36, 1.99, and 1.72 for each respective period 0.33s to 1.49s.

In contrast, we find that CARTEL exhibits no network timeout failures (implying no consecutive network timeout failures) across all items during all 21,000 cases, which is interpreted as follows. First, the experiment confirms achieving our goal: no consecutive network timeout failures under failover environments where the leader node failure occurs, which is *guaranteed* by Theorem 4. Second, although CARTEL does not guarantee the absence of any single network timeout failure, the empirical results show that such failures are rare, with none occurring in this experiment.

Scalability. We study how the size of the buffer grows as the cluster size n increases by conducting a simulation. Each of the n nodes in the cluster generates and submits data every $T_{\phi_x}=200\text{ms}$. The network delay between nodes is randomly chosen within 1–8ms. The election timeout is set to 150–300ms, the pending timeout to 20ms, and the heartbeat interval to 5 ms. The simulation runs for 30 seconds, during which the leader node is randomly failed. In CARTEL, a follower clears its buffer whenever a valid `AppendEntries_RPC` arrives. During steady operation, the largest preflush level within a single heartbeat window serves as an upper bound on transient occupancy. We therefore instrument each node to record the buffer size just before every `AppendEntries_RPC` heartbeat-triggered flush.

The size of the cluster is increased from 50 to 2,000 nodes, and we measure how the buffering demand evolves. We focus on two key metrics: (i) the mean buffer occupancy across followers, which captures the typical load per node, and (ii) the maximum buffer occupancy observed among all followers, which reflects the worst-case stress in the system.

As expected, as illustrated in Figure 9(b), both metrics grow roughly linearly with the number of nodes. At 50 nodes, the average occupancy is approximately 123 entries, and the worst-case occupancy reaches 555. With 100 nodes, the average increases to 249, and the maximum to 996. At 500 nodes, these values rise further to 1,256 and 5,672, respectively. At the largest scale of 2,000 nodes, the system exhibits an average of about 5,000 entries per follower buffer and a worst case of nearly 20,098. This trend shows that while buffering naturally grows with scale, the increase is predictable

and remains within manageable upper bounds for practical deployments, as analyzed in Section IV-E. While the bound in Section IV-E is conservative and not strictly necessary to interpret Figure 9(b), we recommend using it as a deployment-time capacity cap for hard real-time settings; otherwise, it is sufficient to provision to a high percentile (e.g., p99) of the empirical peak with a modest safety margin.

Overall, these experiments demonstrate that CARTEL scales to thousands of nodes while keeping both the average and worst-case buffer occupancy proportional to the system size.³

Network Delay. We consider latencies with one-way link delays $\delta_{X,Y} \in [\delta_{\min}, \delta_{\max}]$ and enforce CARTEL’s no-split constraint $2 \cdot \delta^* \leq \text{pending timeout} < ET^{\min} - 2 \cdot \delta^*$. A follower opens a single pending window upon the first `VoteRequest_RPC` arrival and votes for the smallest- ET candidate it observed during that window.

Table I reports the mean leader-recovery latency (denoted by t_{elect}) in this small-delay regime. In an environment like the scalability simulation, we measure t_{elect} from failure to the instant the to-be-elected candidate actually receives the majority($> n/2$) votes (i.e., the candidate knows it is the leader). Consistent with our analysis, t_{elect} is dominated by $ET^{\min} + \text{pending timeout}$, while network delay contributes only as an *additive* term bounded by $2 \cdot \delta^*$. Consequently, pushing delays into the millisecond range neither destabilizes CARTEL nor inflates failover time. We evaluate three small-delay configurations, specified as $(ET_{\min} - ET_{\max} / \text{pending timeout} / \delta_{\min} - \delta_{\max})$ in milliseconds: (150–300 / 20 / 1–8), (200–400 / 30 / 5–15), and (400–800 / 40 / 10–20). For each case of 500 trials, the mean leader-recovery latency t_{elect} is 177.8ms, 248.1ms, and 469.4ms, respectively. As expected, the means increase monotonically with the election timeout range, and in all cases remain below the bound $t_{\text{elect}} \leq ET_{\min} + \text{pending timeout} + 2 \cdot \delta^*$. In all cases, we observed no split votes, confirming that CARTEL remains safe and provides predictable, low failover latency even under data center/edge-like delays that are much smaller than those used in our prior evaluation.

TABLE I: Leader recovery latencies in the cases of small-delay regime (ms)

$ET_{\min}-ET_{\max}$	pending	$\delta_{\min}-\delta_{\max}$	t_{elect} mean
150–300	20	1–8	177.8
200–400	30	5–15	248.1
400–800	40	10–20	469.4

VI. RELATED WORK

Prior work has extended Raft for improved reliability and fault tolerance in various environments [16]–[19], and broader

³Python-based implementation of CARTEL is publicly accessible at the following repository: <https://github.com/SeungHoon00/CARTEL-Consensus-Adapting-Real-Time-and-Efficient-Logging.git>

studies have explored adaptive failure handling in distributed systems [20]–[23]. However, these approaches primarily focus on availability or consistency, without addressing timing guarantees under failure.

Several recent studies have sought to improve Raft’s leader election process to reduce failover time. NuRaft [14] introduces a priority-based mechanism in which higher-priority nodes are favored during elections, thereby reducing the likelihood of less capable nodes becoming leaders. While this approach can improve average leader election time, it cannot fully eliminate split votes and provides no formal bound on the election completion time. Choumas and Korakis [15] propose an adaptive election timeout strategy that analytically tunes timeouts based on network conditions to minimize the expected election latency. However, this method retains randomized timeouts and thus offers no deterministic guarantee on the worst-case election duration. In contrast, CARTEL eliminates split votes through its pending timeout mechanism and provides a provable upper bound on leader election time, making it suitable for deployment in time-critical distributed systems where probabilistic guarantees are insufficient.

Recent research has increasingly emphasized the need for predictable and resilient consensus mechanisms in real-time and embedded systems. In-ConcReTeS [24] revisits interactive consistency under Byzantine fault models and proposes RT-EIGByz, a time-aware, leaderless protocol suitable for periodic real-time applications. While it provides temporal predictability and is well-suited for fully synchronous environments, its consensus mechanism is designed around all-to-all message exchanges and assumes precise clock synchronization. TinyBFT [25], on the other hand, demonstrates the feasibility of performing Byzantine fault-tolerant state-machine replication on severely resource-constrained embedded devices, such as microcontrollers with less than 1MB of RAM. However, it does not address timing guarantees for specific consensus stages like leader election or commit, and it remains anchored in PBFT’s multi-phase structure and Byzantine assumptions. In contrast, CARTEL extends Raft for partially synchronous systems with a focus on timing predictability rather than Byzantine resilience or minimal memory usage. CARTEL introduces timing-aware vote stabilization and node-side buffering to achieve bounded-time leader election and log commit, even in the presence of transient faults. This makes it particularly suitable for distributed systems that demand time-predictable behavior for safety-critical coordination.

VII. CONCLUSION

This paper introduces CARTEL, a novel consensus algorithm that integrates timing guarantees into the Raft framework, addressing the critical need for timing predictability and data integrity in distributed systems. CARTEL enhances the Raft protocol by implementing CARTEL voting and node buffer mechanisms, which together ensure timely leader election and reduce the number of data not committed in time during leader transitions. Our theoretical analysis and experiments

demonstrate that CARTEL significantly reduces the uncertainty inherent in the leader election process of the standard Raft algorithm. We validated the practical performance of CARTEL through implementation on a cluster of edge devices. The results confirm that CARTEL maintains timing guarantees and mitigate the effect of network timeout failure, highlighting its suitability for deployment in distributed systems where both timing and data integrity are paramount.

ACKNOWLEDGEMENT

This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (RS-2024-00438248, RS-2025-17622972, RS-2025-22502968). The authors appreciate the helpful comments provided by Prof. Kang G. Shin at University of Michigan.

REFERENCES

- [1] D. Ongaro and J. Ousterhout, “In search of an understandable consensus algorithm,” in *2014 USENIX annual technical conference (USENIX ATC 14)*, 2014, pp. 305–319.
- [2] C. Lu, A. Saifullah, B. Li, M. Sha, H. Gonzalez, D. Gunatilaka, C. Wu, L. Nie, and Y. Chen, “Real-time wireless sensor-actuator networks for industrial cyber-physical systems,” *Proceedings of the IEEE*, vol. 104, no. 5, pp. 1013–1024, 2015.
- [3] Y. Zhou, B. Rao, and W. Wang, “UAV swarm intelligence: Recent advances and future trends,” *IEEE Access*, vol. 8, pp. 183 856–183 878, 2020.
- [4] P. Jittrapirom, V. Caiati, A. M. Feneri, S. Ebrahimigharehbaghi, M. J. Alonso-González, and J. Narayan, “Mobility as a service: A critical review of definitions, assessments of schemes, and key challenges,” *Urban Planning*, vol. 2, no. 2, pp. 13–25, 2017.
- [5] R. Jacob, M. Zimmerling, P. Huang, J. Beutel, and L. Thiele, “End-to-end real-time guarantees in wireless cyber-physical systems,” in *Proceedings of IEEE Real-Time Systems Symposium (RTSS)*, 2016, pp. 167–178.
- [6] R. Kumar, M. Hasan, S. Padhy, K. Evchenko, L. Piramanayagam, S. Mohan, and R. B. Bobba, “End-to-end network delay guarantees for real-time systems using sdn,” in *Proceedings of IEEE Real-Time Systems Symposium (RTSS)*, 2017, pp. 231–242.
- [7] K. Agrawal, S. Baruah, Z. Guo, J. Li, and S. Vaidhun, “Hard-real-time routing in probabilistic graphs to minimize expected delay,” in *Proceedings of IEEE Real-Time Systems Symposium (RTSS)*, 2020, pp. 63–75.
- [8] E. Mohammadpour, E. Stai, and J.-Y. Le Boudec, “Improved delay bound for a service curve element with known transmission rate,” *IEEE Networking Letters*, vol. 1, no. 4, pp. 156–159, 2019.
- [9] L. Thomas, J.-Y. Le Boudec, and A. Mifdaoui, “On cyclic dependencies and regulators in time-sensitive networks,” in *Proceedings of IEEE Real-Time Systems Symposium (RTSS)*, 2019, pp. 299–311.
- [10] C. Dwork, N. Lynch, and L. Stockmeyer, “Consensus in the presence of partial synchrony,” *Journal of the ACM (JACM)*, vol. 35, no. 2, pp. 288–323, 1988.
- [11] D. Ganesan, S. Ratnasamy, H. Wang, and D. Estrin, “Coping with irregular spatio-temporal sampling in sensor networks,” *ACM SIGCOMM Computer Communication Review*, vol. 34, no. 1, pp. 125–130, 2004.
- [12] D. Natale *et al.*, “Dynamic end-to-end guarantees in distributed real time systems,” in *Proceedings of IEEE Real-Time Systems Symposium (RTSS)*, 1994, pp. 216–227.
- [13] G.-X. Liu, L.-F. Shi, and D.-J. Xin, “Data integrity monitoring method of digital sensors for internet-of-things applications,” *IEEE Internet of Things Journal*, vol. 7, no. 5, pp. 4575–4584, 2020.
- [14] “Leader election priority in nuraft,” https://github.com/eBay/NuRaft/blob/master/docs/leader_election_priority.md, accessed: 2025-08-11.

- [15] K. Choumas and T. Korakis, "On using raft over networks: Improving leader election," *IEEE Transactions on Network and Service Management*, vol. 19, no. 2, pp. 1129–1141, 2022.
- [16] Y. Li, Y. Fan, L. Zhang, and J. Crowcroft, "Raft consensus reliability in wireless networks: Probabilistic analysis," *IEEE Internet of Things Journal*, vol. 10, no. 14, pp. 12 839–12 853, 2023.
- [17] D. Huang, X. Ma, and S. Zhang, "Performance analysis of the raft consensus algorithm for private blockchains," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 50, no. 1, pp. 172–181, 2019.
- [18] R. Wang, L. Zhang, Q. Xu, and H. Zhou, "K-bucket based raft-like consensus algorithm for permissioned blockchain," in *Proceedings of IEEE International Conference on Parallel and Distributed Systems (ICPADS)*, 2019, pp. 996–999.
- [19] Y. Wang, S. Li, L. Xu, and L. Xu, "Improved raft consensus algorithm in high real-time and highly adversarial environment," in *Proceedings of Springer Web Information Systems and Applications (WISA)*, 2021, pp. 718–726.
- [20] S. Malik and F. Huet, "Adaptive fault tolerance in real time cloud computing," in *Proceedings of IEEE World Congress on services*, 2011, pp. 280–287.
- [21] V. Venkatesan and I. Iliadis, "A general reliability model for data storage systems," in *Proceedings of IEEE International conference on quantitative evaluation of systems*, 2012, pp. 209–219.
- [22] J. Li, P. Li, R. J. Stones, G. Wang, Z. Li, and X. Liu, "Reliability equations for cloud storage systems with proactive fault tolerance," *IEEE Transactions on Dependable and Secure Computing*, vol. 17, no. 4, pp. 782–794, 2018.
- [23] F. S. Abkenar and A. Jamalipour, "A reliable data loss aware algorithm for fog-iot networks," *IEEE Transactions on Vehicular Technology*, vol. 69, no. 5, pp. 5718–5722, 2020.
- [24] A. Gujarati, N. Yang, and B. B. Brandenburg, "In-ConcReTeS: Interactive consistency meets distributed real-time systems, again!" in *Proceedings of the 43rd IEEE Real-Time Systems Symposium (RTSS)*, 2022, pp. 211–224.
- [25] H. Böhm, T. Distler, and P. Wägemann, "TinyBFT: Byzantine fault-tolerant replication for highly resource-constrained embedded systems," in *Proceedings of the 30th IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2024, pp. 225–238.